

Week 9

Information Extraction

Guan-Yun Wang

Department of Information Management

Fu Jen Catholic University

2026.04.24

Introduction

This chapter will cover the basics of information extraction. Information extraction is the task of pulling very specific information from text.

- For example, if you want to know the companies mentioned in a news article, you can use information extraction techniques to access the companies.

Topics:

- Using regular expressions
- Finding similar strings - Levenshtein distance
- Extracting keywords
- Performing named entity recognition using spaCy
- Training your own NER model with spaCy
- Fine-tuning BERT for NER

Using regular expressions

- In this example, we will use regular expressions to find email addresses and URLs in text.
- Regular expressions are special character sequences that define search patterns and can be created and used via the Python `re` package.

Getting ready

- job descriptions dataset:
<https://www.kaggle.com/datasets/andrewmvd/data-scientist-jobs>



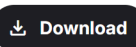
LARXEL · UPDATED 6 YEARS AGO



111



Code



Download



Data Scientist Jobs

3909 job listing for data scientist roles scraped from glassdoor



Data Card

Code (73)

Discussion (1)

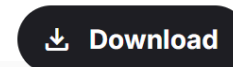
Suggestions (0)



111



Code



Download



DOWNLOAD VIA

kagglehub

New to Kaggle API? Here's how to [set up your API keys.](#)

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("andrewmvd/data-scientist-jobs")

print("Path to dataset files:", path)
```

Implementation

1. import the `re` and `pandas` packages:

```
import re
import pandas as pd
```

2. Read in the data and check the contents inside it:

```
data_file = "../data/DataScientist.csv"
df = pd.read_csv(data_file, encoding='utf-8')
print(df)
```

The output will be long and should start like this:

```

Unnamed: 0  index                                Job Title \
0           0           0                        Senior Data Scientist
1           1           1      Data Scientist, Product Analytics
2           2           2                        Data Science Manager
3           3           3                        Data Analyst
4           4           4      Director, Data Science
...         ...         ...
3904        3904      4375                        AWS Data Engineer
3905        3905      4376      Data Analyst â Junior
3906        3906      4377      Security Analytics Data Engineer
3907        3907      4378      Security Analytics Data Engineer
3908        3908      4379  Patient Safety Physician or Safety Scientist -...

```

```

                                Salary Estimate \
0      $111K-$181K (Glassdoor est.)
1      $111K-$181K (Glassdoor est.)
2      $111K-$181K (Glassdoor est.)
3      $111K-$181K (Glassdoor est.)
4      $111K-$181K (Glassdoor est.)
...         ...
3904    $55K-$112K (Glassdoor est.)
3905    $55K-$112K (Glassdoor est.)
3906    $55K-$112K (Glassdoor est.)
3907    $55K-$112K (Glassdoor est.)
3908    $55K-$112K (Glassdoor est.)
...
3907  Less than $1 million (USD)                -1                -1
3908    $2 to $5 billion (USD)                AbbVie                -1

```

3. The `get_list_of_items` helper function takes a DataFrame as input and turns one of its columns into a list. It accepts the DataFrame and the column name as inputs.

```
def get_list_of_items(df, column_name):  
    values = df[column_name].values # get the column values  
    values = [item for sublist in values for item in sublist] #flatten the list  
    list_of_items = list(set(values)) # remove duplicates by turning the list into a set.  
    return list_of_items
```

4. In this step, we define the `get_emails` function to get all the emails that appear in the `Job Description` column. The regular expression consists of three parts that appear in square brackets followed by quantifiers.

```
def get_emails(df):  
    email_regex = '^[^s:|()\' ]+@[a-zA-Z0-9\.\.]+\.[a-zA-Z]+'  
    df['emails'] = df['Job Description'].apply(lambda x: re.findall(email_regex, x))  
    emails = get_list_of_items(df, 'emails')  
    return emails
```

- `[^\\s:|()\\']+` is the username part of the regular expression, followed by the @ sign.
 - `[]` is for "group of characters"
 - The characters in the username can be anything but a space (`\\s`), colon(`:`), pipe(`|`), and apostrophe(`'`). (An apostrophe is a special character in regular expressions and has to be escaped with a backward slash in order to invoke the regular meaning of the character.)
 - The `^` character shows the negation of the character class.
- `[a-zA-Z0-9\\.]+` is the first part of the domain name, followed by a dot.
 - This part is simply alphanumeric characters, lowercase or uppercase, and a dot appearing one or more times.
- `[a-zA-Z]+` is the last part of the domain name, which is the top-level domain, such as `.com`, `.org`. Usually, no digits are allowed in these top-level domains.

5. We will now get the emails from the DataFrame using the previous functions:

```
emails = get_emails(df)
print(emails)
```

Part of the results will look like this:

```
['Melissa@kingmelissa.com', 'neeraj.h@avacend.com',  
'hr@cloudflare.com', 'bwaldron@apexsystems.com',  
'CareersUS@QVC.com', 'kushal@nextgentechologiesinc.com',  
'human.resources@thrivent.com', 'DisabilityAccommodations@catalent.com',  
'rohit.mcdonald@prolim.com', 'employeeservices@apexsystemsinc.com',  
'Aanvesh@conchtech.com', .....]
```


6. The `get_urls` helper function takes a DataFrame as input and turns one of its columns into a list.

```
def get_urls(df):
    url_regex = '(\http[s]?://(www\.)?[A-Za-z0-9-\_\.\-]+\.[A-Za-z]+/?[A-Za-z0-9$_-_\-\./\.\.]*(\.\.)\"*)*'
    df['urls'] = df['Job Description'].apply(lambda x: [x[item.span()[0]:item.span()[1]] for item in re.finditer(url_regex, x)])
    urls = get_list_of_items(df, 'urls')
    return urls
```

- `http[s]?://` This is the `http` part of the URL.
- `(www\.)?` this is the `www` part of the URL, followed by a dot, escaped with a backward slash.
- `[A-Za-z0-9-\._\~]+` This is the domain name of the website.
- `[A-Za-z]+/?[A-Za-z0-9$\-\_\~\!/\.\']*+` This part is whatever follows the domain name after the slash.
 - They could or could not be present, and that is why they are followed by the `*` quantifier.
- `(\.\.)\""]*` Many URLs in this dataset are followed by dots, brackets, and other characters, and this is the last part of the regular expression.

Explore Regular Expression

- You can use regular expression testing websites to enter the text in which you expect a match and the regular expression:
<https://regex101.com/>
- You can also ask LLM (chatGPT, Gemini, etc) to help you write a regular expression for you.

Finding Similar Strings - Levenshtein Distance

- When doing information extraction, in many cases, we deal with misspellings, which can bring complications to the task.
- To get around this problem, several methods are available, including Levenshtein distance.

Preparation

We will use python- `Levenshtein` package.

In your terminal or your notebook:

```
!pip install levenshtein
```

Implementation

- Do the necessary imports:

```
import pandas as pd
import Levenshtein
```

- The `find_levenshtein` function takes in a DataFrame and an input string and computes the Levenshtein distance between it and each string in the emails column. It takes in an input string and a DataFrame with emails and creates a new column in which the value is the Levenshtein distance between the input and the email address in the DataFrame.

```
def find_levenshtein(input_string, df):
    df['distance_to_' + input_string] = \
        df['emails'].apply(lambda x: Levenshtein.distance(input_string, x))
    return df
```

- We define the `get_closest_email_lev` function, which takes in a DataFrame with emails and an email to match and returns the email in the DataFrame that is closest to the input. We also create a new column with distances to the input email and then using the `idxmin()` function from `pandas` to find the index of the minimum value. (minimum index help us find the closest email.)

```
def get_closest_email_lev(df, email):  
    df = find_levenshtein(email, df)  
    column_name = 'distance_to_' + email  
    minimum_value_email_index = df[column_name].idxmin()  
    email = df.loc[minimum_value_email_index]['emails']  
    return email
```

Here you can test the misspelled email address to find a match in the new `email` DataFrame.

```
new_df = pd.DataFrame(emails, columns=['emails'])
input_string = "rohitt.macdonald@prelim.com"
email = get_closest_email_lev(new_df, input_string)
print(email)
```

Furthermore, we can also use **Jaro distance** to calculate the similarity between two strings.

```
def find_jaro(input_string, df):  
    df['distance_to_' + input_string] = df['emails'].apply(lambda x: Levenshtein.jaro(input_string, x))  
    return df
```

But if you use jaro, we need to **find the maximum** to find the similar target.

```
def get_closest_email_jaro(df, email):  
    df = find_jaro(email, df)  
    column_name = 'distance_to_' + email  
    maximum_value_email_index = df[column_name].idxmax()  
    email = df.loc[maximum_value_email_index]['emails']  
    return email
```

```
email = get_closest_email_jaro(new_df, input_string)  
print(email)
```

Extracting keywords

Here, we will be working with the BBC news dataset that contains news articles. (We have used it in the chapter of classification)

- **Extracting keywords from text can give us a quick idea about what the article is about** and can also serve as a basis for a **tagging system**, for example, on a website.
- For the extraction to work correctly, we need to train a TF-IDF vectorizer that we will use during the extraction phase.

Implementation

- Import the necessary packages and functions:

```
from datasets import load_dataset
from nltk import word_tokenize
from math import ceil
from sklearn.feature_extraction.text import TfidfVectorizer
from nltk.corpus import stopwords
```

- Load the training and testing datasets, convert them to `pandas` DataFrame objects, and print out the training DataFrame to discover how it looks.

```
train_dataset = load_dataset("SetFit/bbc-news", split="train")
test_dataset = load_dataset("SetFit/bbc-news", split="test")
train_df = train_dataset.to_pandas()
test_df = test_dataset.to_pandas()
print(train_df)
print(test_df)
```

- Create the vectorizer and fit it on the training data text.
- Here, we use English stopwords, a minimum document frequency of 2, and a maximum document frequency of 95%.

```
vectorizer = TfidfVectorizer(stop_words='english', min_df=2, max_df=0.95)
vectorizer.fit(train_df["text"])
```

We also define a helper functions. This will sort a **coordinate matrix** by the TF-IDF score. It takes the coordinate matrix that is converted from the vector created by the vectorizer.

```
def sort_data_tfidf_score(coord_matrix):
    # Create a list of tuples from the data (index, TF-IDF score)
    tuples = zip(coord_matrix.col, coord_matrix.data)
    # sort the tuple list by the TF-IDF score and returns the sorted results.
    return sorted(tuples, key=lambda x: (x[1], x[0]), reverse=True)
```

The `get_keyword_strings` function will get the keywords for a given vector.

```
def get_keyword_strings(vectorizer, num_words, sorted_vector):  
    words = []  
    index_dict = vectorizer.get_feature_names_out()  
    for (item_index, score) in sorted_vector[0:num_words]:  
        word = index_dict[item_index]  
        words.append(word)  
    return words
```

The `get_keywords_simple` function will return a list of keywords for a given text.

```
def get_keywords_simple(vectorizer, input_text, num_output_words=10):  
    vector = vectorizer.transform([input_text])  
    sorted = sort_data_tfidf_score(vector.tocoo())  
    words = get_keyword_strings(vectorizer, num_output_words, sorted)  
    return words
```

Here we can use the previous functions to test.

```
print(test_df.iloc[0]["text"]) # The first article
keywords = get_keywords_simple(vectorizer, test_df.iloc[0]["text"])
print(keywords)
```

Can you identify the keywords in this article?

There's more...

If you want to extract the keywords which are not only individual words but also phrases, you will use a more sophisticated approach.

Use the parameter: `ngram_range`

```
stop_words = list(stopwords.words('english'))
stop_words.remove("the")
trigram_vectorizer = TfidfVectorizer(stop_words=stop_words, min_df=2, ngram_range=(1,3), max_df=0.95)
trigram_vectorizer.fit(train_df["text"])
```

The `get_keyword_strings_all` function will get all the keywords from the sorted vector, and has no restriction on how many words it gets.

```
def get_keyword_strings_all(vectorizer, sorted_vector):  
    words = []  
    index_dict = vectorizer.get_feature_names_out()  
    for (item_index, score) in sorted_vector:  
        word = index_dict[item_index]  
        words.append(word)  
    return words
```

The `get_keywords_complex` function outputs main keywords and phrases up to three words long.

```
def get_keywords_complex(vectorizer, input_text, spacy_model, num_words=70):
    keywords = []
    doc = spacy_model(input_text)
    vector = vectorizer.transform([input_text])
    sorted = sort_data_tfidf_score(vector.tocoo())
    ngrams = get_keyword_strings_all(vectorizer, sorted)
    ents = [ent.text.lower() for ent in doc.noun_chunks]
    for i in range(0, num_words):
        keyword = ngrams[i]
        if keyword.lower() in ents and not keyword.isdigit() and keyword not in keywords:
            keywords.append(keyword)
    return keywords
```

Test your results:

```
keywords = get_keywords_complex(trigram_vectorizer, test_df.iloc[0]["text"], small_model)
print(keywords)
```

Named Entity Recognition (命名實體識別)

Named entity recognition (NER) is the task of parsing the names of places, people, organizations, and so on, out of text.

- This can be useful in many downstream tasks.

Implementation

Initialize the article text.

```
article = ""iPhone 12: Apple makes jump to 5G
Apple has confirmed its iPhone 12 handsets will be its first to work on faster 5G networks.
The company has also extended the range to include a new "Mini" model that has a smaller 5.4in screen.
The US firm bucked a wider industry downturn by increasing its handset sales over the past year.
But some experts say the new features give Apple its best opportunity for growth since 2014, when it revamped its line-up with the iPhone 6.
"5G will bring a new level of performance for downloads and uploads, higher quality video streaming, more responsive gaming,
real-time interactivity and so much more," said chief executive Tim Cook.
There has also been a cosmetic refresh this time round, with the sides of the devices getting sharper, flatter edges.
The higher-end iPhone 12 Pro models also get big.....
```


Create the spacy `Doc` object and use it to extract the entities. The `Doc` object is created by using the small spaCy model on the text.

```
doc = small_model(article)    # you can also use large model to do the same thing
print(len(doc.ents))
small_model_ents = doc.ents
for ent in doc.ents:
    print(ent.text, ent.start_char, ent.end_char, ent.label_)
```

The output: (The `NORP` entity stands for Nationalities or Religious or Political groups)

```
46
12 7 9 CARDINAL
Apple 11 16 ORG
5 31 32 CARDINAL
...
2.7% 2594 2598 PERCENT
Chinese 2652 2659 NORP
Sina 2797 2801 ORG
```

Training your own NER model with spaCy

We can use the pretrained spaCy model to extract named entities.
But how should we do when we want to create a new one from scratch?

Here we take music NER into consideration

Music NER

<https://github.com/deezer/music-ner-eacl2023>

Implementation

import functions and packages

```
import pandas as pd
from spacy.cli.train import train
from spacy.cli.evaluate import evaluate
from spacy.tokens import DocBin
from sklearn.model_selection import train_test_split
```

```
music_ner_df = pd.read_csv('../data/music_ner.csv')
print(music_ner_df)
```

Remove `_deduced` from the labels so the labels are now `Artist`, `WoA (work of art)`, `Artist_or_WoA`

```
def change_label(input_label):  
    input_label = input_label.replace("_deduced", "")  
    return input_label  
  
music_ner_df["label"] = music_ner_df["label"].apply(change_label)  
print(music_ner_df)
```

Create the `DocBin` objects that will store the processed data which can be the input data for spaCy models.

```
train_db = DocBin()  
test_db = DocBin()
```

Create a list of unique IDs and split it into training and test data.

```
# Get a unique list of unique ids
ids = list(set(music_ner_df["id"].values))
print(len(ids))
# Split ids into training and test
train_ids, test_ids = train_test_split(ids)
print(len(train_ids))
print(len(test_ids))
```

Create and save training and testing data in the `DocBin` objects.

```
# Go through the list of ids and get all the rows associated with each id
for id in ids:
    entity_rows = music_ner_df.loc[music_ner_df['id'] == id]
    text = entity_rows.head(1)["text"].values[0]
    doc = small_model(text) # You can change to large model
    ents = []
    for index, row in entity_rows.iterrows():
        label = row["label"]
        start = row["start_offset"]
        end = row["end_offset"]
        span = doc.char_span(start, end, label=label, alignment_mode="contract")
        ents.append(span)
    doc.ents = ents
    if id in train_ids:
        train_db.add(doc)
    else:
        test_db.add(doc)
train_db.to_disk('../data/music_ner_train.spacy')
test_db.to_disk('../data/music_ner_test.spacy')
```

Here we're going to train the model.

- Use the `spacy_config_ner.cfg` configuration file in the folder. (download from Tronclass)
- The output shows the loss, accuracy, precision, recall, F1 score, and other metrics for every epoch.

```
# Train the model  
train("./spacy_config_ner.cfg", output_path="../models/spacy_music_ner")
```

In this step, we load the trained model and use it on data not seen during training.

```
# Use the trained model for prediction
nlp = spacy.load("../models/spacy_music_ner/model-last")
first_test_id = test_ids[0] #get an ID from the test set
test_rows = music_ner_df.loc[music_ner_df['id'] == first_test_id] # get all the rows from the test data
input_text = entity_rows.head(1)["text"].values[0]
print(input_text)
print("====Gold entities:====")
for index, row in entity_rows.iterrows():
    label = row["label"]
    start = row["start_offset"]
    end = row["end_offset"]
    span = doc.char_span(start, end, label=label, alignment_mode="contract")
    print(span)
doc = nlp(input_text)
print("====Predicted entities: ====")
for entity in doc.ents:
    print(entity)
```


Results check

Can you identify the singers and the songs?

Evaluate the model

We can evaluate the model by using spaCy's `evaluate` function.

```
# Evaluate the model
evaluate('../models/spacy_music_ner/model-last', '../data/music_ner_test.spacy')
```

Fine-tune with BERT model

(check it out in your student notebook)